

Approximating the minmax rooted-tree cover in a tree

Hiroshi Nagamochi^{a,*}, Kohei Okada^b

^a Department of Applied Mathematics and Physics, Kyoto University, Yoshida Honmachi, Sakyo, Kyoto 606-8501, Japan

^b Matsushita Electric Industrial Co., Ltd., Kadoma 1006, Osaka 571-8501, Japan

Received 15 November 2005; received in revised form 12 April 2006

Available online 22 June 2007

Communicated by F.Y.L. Chin

Abstract

Given an edge-weighted rooted tree T and a positive integer p ($\leq n$), where n is the number of vertices in T , we cover all vertices in T by a set of p subtrees each of which contains the root r of T . The minmax rooted-tree cover problem asks to find such a set of p subtrees so as to minimize the maximum weight of the subtrees in the set. In this paper, we propose an $O(n)$ time $(2 + \varepsilon)$ -approximation algorithm to the problem, where $\varepsilon > 0$ is a prescribed constant.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Approximation algorithms; Graph algorithms; Tree partitioning; Tree covers; Combinatorial problems; Design of algorithms

1. Introduction

Let $G = (V, E)$ be a simple undirected graph with a vertex set V and an edge set E such that each edge e is weighted by a positive real $w(e)$. The vertex set and the edge set of G may be denoted by $V(G)$ and $E(G)$, respectively. Let $n = |V(G)|$ and $m = |E(G)|$. A *subtree* of G is a connected subgraph of G . A collection $\mathcal{X}$ of disjoint subsets $X_1, X_2, \dots, X_k$ of V is called a *partition* of V if their union is V , where some X_i may be empty. A collection $\mathcal{X}$ of V is called a p -*partition* of V if $|\mathcal{X}| = p$. For a subgraph H of G , we denote $\sum_{e \in E(H)} w(e)$ by $w(H)$. For two vertices u and v , the weight of a shortest path (with respect to the edge weight w) between u and v in G is denoted by

$dist_G(u, v)$. For a tree T rooted at a vertex r , the set of leaves (i.e., non-root vertices of degree 1) is denoted by $L(T)$. For a subset $X \subseteq V(T)$ of vertices, let $T\langle X \rangle$ denote the minimal subtree of T that contains X (where the leaves of $T\langle X \rangle$ will be vertices in X). For a set A and an element $a \notin A$, we may denote set $A \cup \{a\}$ by $A + a$.

Given a graph, the problem of finding a set of tours or trees that cover all vertices in the graph so as to minimize an objective function arises in numerous applications such as the multi-vehicle scheduling problem [3,5–7], and the task sequencing problem [2]. The following problem has been studied in [1,8].

Minmax Rooted-Tree Cover Problem (MRSC)

Input: An instance $I = (G, w, p)$ which consists of a connected graph G with a designated vertex $r \in V(G)$ called a root, an edge-weight w , and an integer $p \in [2, n]$, where $n = |V(G)|$.

* Corresponding author.

E-mail addresses: nag@amp.i.kyoto-u.ac.jp (H. Nagamochi), okada.kohei@jp.panasonic.com (K. Okada).

Feasible solution: A set $\mathcal{T}$ of p subtrees $T_1, T_2, \dots, T_p$ of G such that $r \in V(T_i)$ ($1 \leq i \leq p$) and $\bigcup_{1 \leq i \leq p} V(T_i) = V(G)$.

Goal: Minimize $w_{\max}(\mathcal{T}) := \max_{1 \leq i \leq p} w(T_i)$.

For the MRSC with a general graph G , a $(3 - 2/(p + 1))$ -approximation algorithm is obtained in [8]. Notice that the MRSC is NP-hard even if G is a tree and $p = 2$, since it contains PARTITION [4] as its special case. In the case where G is a tree, a linear time $4/3$ -approximation algorithm is known for $p = 2$ [1]. In the article [3], a 4-approximation algorithm is proposed to a related minmax tree cover problem, where p specified roots $r_1, r_2, \dots, r_p$ in G are given and the objective is to find p subtrees T_i rooted at r_i ($i = 1, 2, \dots, p$) minimizing the maximum weight in the subtrees. In this paper, we propose a $(2 + \varepsilon)$ -approximation algorithm to the MRSC with a tree G and $p \geq 2$, where $\varepsilon > 0$ is a prescribed constant.¹ For this, we introduce a *reverse problem* of the MRSC as follows.

We easily observe that any instance $I = (T, w, p)$ of the MRSC, where T is a tree, has an optimal solution $\mathcal{T} = \{T_1, T_2, \dots, T_p\}$ which is given by a p -partition $\mathcal{L} = \{L_1, L_2, \dots, L_p\}$ of $L(T)$ in such a way that each tree $T_i \in \mathcal{T}$ is obtained by $T_i = T \langle L_i + r \rangle$, $i = 1, 2, \dots, p$. We treat a p -partition $\mathcal{L}$ of $L(T)$ as a solution to $I = (T, w, p)$, and define

$$w_{\max}(\mathcal{L}) = \max_{L_i \in \mathcal{L}} w(T \langle L_i + r \rangle) \quad \text{and}$$

$$w_{\text{sum}}(\mathcal{L}) = \sum_{L_i \in \mathcal{L}} w(T \langle L_i + r \rangle).$$

For an edge-weighted rooted tree (T, w) and a real $x > 0$, let $p^*(T, w, x)$ denote the least integer p such that there exists p -partition $\mathcal{L}$ of $L(T)$ such that $w_{\max}(\mathcal{L}) \leq x$. Let $w_{\text{sum}}^*(T, w, x)$ denote the minimum of $w_{\text{sum}}(\mathcal{L})$ over all partitions $\mathcal{L}$ of $L(T)$ such that $w_{\max}(\mathcal{L}) \leq x$.

Bicriteria Rooted-Tree Cover Problem (BRSC)

Input: An instance (T, w, x) which consists of a tree T rooted at a vertex $r \in V(T)$, an edge-weight w , and a real $x > 0$.

Feasible solution: A p -partition $\mathcal{L} = \{L_1, L_2, \dots, L_p\}$ of $L(T)$ such that $p \leq p^*(T, w, x)$.

Goal: Minimize $\max\{w_{\max}(\mathcal{L})/x, w_{\text{sum}}(\mathcal{L})/w_{\text{sum}}^*(T, w, x)\}$.

We prove that there is a 2-approximation algorithm to BRSC, which delivers a 2-approximate solution $\mathcal{L}$ for both criteria $w_{\max}(\mathcal{L})/x$ and $w_{\text{sum}}(\mathcal{L})/w_{\text{sum}}^*(T, w, x)$.

2. Lower bounds

For an instance $I = (T, w, p)$ of the MRSC, denote by $w_{\max}^*(I)$ or $w_{\max}^*(T, w, p)$ the optimal value to $I = (T, w, p)$, i.e., the minimum of $w_{\max}(\mathcal{L})$ over all p -partitions $\mathcal{L} = \{L_1, L_2, \dots, L_p\}$ of $L(T)$. This section derives lower bounds on $w_{\max}^*(T, w, p)$, $p^*(T, w, x)$ and $w_{\text{sum}}^*(T, w, x)$.

Let T be a tree rooted at a vertex r . Let $Ch(v)$ denote the set of children of a vertex v . For a subset $X \subseteq V(T)$, let $\text{lca}(X) \in V(T)$ denote the least common ancestor of all vertices in X , and $D(X)$ denote the set of all descendants of the vertices in X (including the vertices in X). For a vertex v and a subset $C' \subseteq Ch(v)$, a subtree $T \langle D(C') + v \rangle$ is called a *subtree rooted at v* . For a vertex u , let $d(u)$ denote the distance of u from r , i.e., $d(u) = \text{dist}_T(r, u)$. Define

$$LB = \max \left\{ \frac{w(T)}{p}, \max_{u \in L(T)} d(u) \right\},$$

which is a lower bound on $w_{\max}^*(I)$.

Lemma 1. $LB \leq w_{\max}^*(I) \leq 3LB$.

Proof. We easily see that LB is a lower bound on $w_{\max}^*(I)$, i.e., $LB \leq w_{\max}^*(I)$. We show $w_{\max}^*(I) \leq 3LB$. For the MRSC with a general graph G , it is known [8] that there is a solution $\mathcal{T}$ such that $w_{\max}(\mathcal{T}) \leq (2 - 2/(p + 1))w(\hat{T})/p + \max_{u \in V(G)} \text{dist}_G(r, u)$ holds for a minimum spanning tree $\hat{T}$ of G . Thus, this implies that, for a tree T , there is a solution $\mathcal{T}$ such that $w_{\max}(\mathcal{T}) \leq (2 - 2/(p + 1))w(T)/p + \max_{u \in L(T)} d(u) \leq 3LB$. Hence $w_{\max}^*(I) \leq 3LB$. $\square$

We next derive lower bounds on $p^*(T, w, x)$ and $w_{\text{sum}}^*(T, w, x)$ by partitioning a subset of $L(T)$ into appropriate subsets. A *subpartition* $\mathcal{L}' = \{L'_1, L'_2, \dots, L'_k\}$ of the set $L(T)$ is a partition of a subset of $L(T)$, and is called *independent* if, for any two L'_i and L'_j ,

$T \langle L'_i + r \rangle$ and $T \langle L'_j + \text{lca}(L'_j) \rangle$ are edge-disjoint

(i.e., no path between r and a leaf $u \in L'_i$ contains any edge in $T \langle L'_j + \text{lca}(L'_j) \rangle$). For a subpartition $\mathcal{L}'$ of $L(T)$, define

$$\hat{p}(\mathcal{L}', T, w) = \sum_{L' \in \mathcal{L}'} \frac{w(T \langle L' + \text{lca}(L') \rangle)}{x - d(\text{lca}(L'))}.$$

¹ A preliminary version of this paper appeared in [9]. This paper extends the result in [9] on the reverse problem to the case where the total weights of subtrees is also to be minimized.

Theorem 2. For an edge-weighted rooted tree (T, w) and a positive real x , let $\mathcal{L}^*$ be a partition of $L(T)$ such that $w(T\langle L^* + r \rangle) \leq x$ for each $L^* \in \mathcal{L}^*$, and $\mathcal{L}'$ be an independent subpartition of $L(T)$ such that $d(\text{lca}(L')) < x$ for each $L' \in \mathcal{L}'$. Then

$$x \hat{p}(\mathcal{L}', T, w) \leq w_{\text{sum}}(\mathcal{L}^*).$$

Proof. Let $\mathcal{L}^* = \{L_i^* \mid i = 1, 2, \dots, p\}$, $\mathcal{L}' = \{L_j' \mid j = 1, 2, \dots, k\}$, $w_i = w(T\langle L_i^* + r \rangle)$, $r_j' = \text{lca}(L_j')$ and $d_j' = d(r_j')$. It suffices to show that

$$x \sum_{1 \leq j \leq k} \frac{w(T\langle L_j' + r_j' \rangle)}{x - d_j'} \leq \sum_{1 \leq i \leq p} w_i.$$

We first derive an upper bound on each $w(T\langle L_j' + r_j' \rangle)$. Let $E_{i,j} = E(T\langle L_i^* + r \rangle) \cap E(T\langle L_j' + r_j' \rangle)$, and $w_{i,j}$ be the sum of the weights of the edges in $E_{i,j}$, where $w_{i,j} = 0$ if $E_{i,j} = \emptyset$. Since $\mathcal{L}^*$ is partition of $L(T)$, all edges in each $T\langle L_j' + r_j' \rangle$ are covered by the subtrees $T\langle L_i^* + r \rangle$, $i = 1, 2, \dots, p$. Hence

$$\sum_{1 \leq i \leq p} w_{i,j} \geq w(T\langle L_j' + r_j' \rangle), \quad 1 \leq j \leq k. \quad (1)$$

We next derive a lower bound on each w_i . For each L_i^* , let $J_i = \{j \mid E_{i,j} \neq \emptyset\}$ and $d_i^* = \max\{d_j' \mid j \in J_i\}$. By the independence of $\mathcal{L}'$, we see that

$$w_i \geq d_i^* + \sum_{j \in J_i} w_{i,j}, \quad 1 \leq i \leq p.$$

From this, $x \geq w_i$ and $x > d_i^*$, it holds

$$\begin{aligned} w_i &\geq \frac{x(w_i - d_i^*)}{x - d_i^*} \geq \frac{x}{x - d_i^*} \sum_{j \in J_i} w_{i,j} \\ &\geq x \sum_{j \in J_i} \frac{w_{i,j}}{x - d_j'}. \end{aligned}$$

Therefore, by (1) we have

$$\begin{aligned} \sum_{1 \leq i \leq p} w_i &\geq x \sum_{1 \leq i \leq p} \sum_{j \in J_i} \frac{w_{i,j}}{x - d_j'} \\ &= x \sum_{1 \leq j \leq k} \frac{\sum_{1 \leq i \leq p} w_{i,j}}{x - d_j'} \\ &\geq x \sum_{1 \leq j \leq k} \frac{w(T\langle L_j' + r_j' \rangle)}{x - d_j'}, \end{aligned}$$

as required. $\square$

Corollary 3. For an edge-weighted rooted tree (T, w) and a positive real x , let $\mathcal{L}'$ be an independent subpartition of $L(T)$ such that $d(\text{lca}(L')) < x$ for each $L' \in \mathcal{L}'$. Then

- (i) $x \hat{p}(\mathcal{L}', T, w) \leq w_{\text{sum}}^*(T, w, x)$.
- (ii) $\hat{p}(\mathcal{L}', T, w) \leq p^*(T, w, x)$.

Proof. (i) Let $\mathcal{L}^*$ be a partition of $L(T)$ such that $w_{\text{sum}}(\mathcal{L}^*) = w_{\text{sum}}^*(T, w, x)$ and $w(T\langle L^* + r \rangle) \leq x$ for each $L^* \in \mathcal{L}^*$. By applying Theorem 2 to $\mathcal{L}'$ and $\mathcal{L}^*$, we have $x \hat{p}(\mathcal{L}', T, w) \leq w_{\text{sum}}(\mathcal{L}^*) = w_{\text{sum}}^*(T, w, x)$, proving (i).

(ii) Let $\mathcal{L}^\dagger$ be a $p^*(T, w, x)$ -partition of $L(T)$ such that $w(T\langle L^\dagger + \text{lca}(L^\dagger) \rangle) \leq x$ for each $L^\dagger \in \mathcal{L}^\dagger$. By applying Theorem 2 to $\mathcal{L}'$ and $\mathcal{L}^\dagger$, we have $x \hat{p}(\mathcal{L}', T, w) \leq w_{\text{sum}}(\mathcal{L}^\dagger) \leq x|\mathcal{L}^\dagger| = xp^*(T, w, x)$, which implies $\hat{p}(\mathcal{L}', T, w) \leq p^*(T, w, x)$. $\square$

3. Algorithms for MRSC and BRSC

This section presents approximation algorithms for the MRSC and BRSC with a tree T . We first introduce the following two operations that modify an edge-weighted tree T rooted at r into another edge-weighted tree T' rooted at r .

- (i) Let T_1 be a subtree of T rooted at a vertex r_1 (i.e., $T_1 = T\langle D(C') + r_1 \rangle$ for some subset $C' \subseteq \text{Ch}(r_1)$). For an ancestor u of r_1 in T , *lifting* T_1 up to u consists of detaching T_1 from r_1 and attaching the T_1 at u , where the weight of each edge in the tree remains unchanged.
- (ii) Let $e = (u, v)$ be the edge incident to a leaf v in T . *Splitting* e by a real $a \in (0, w(e))$ consists of reducing the weight of e to $w'(e) = w(e) - a$ and adding a new edge $e' = (u, v')$ of weight $w'(e') = a$ between u and a new leaf v' , where the weight of any other edge in the tree remains unchanged.

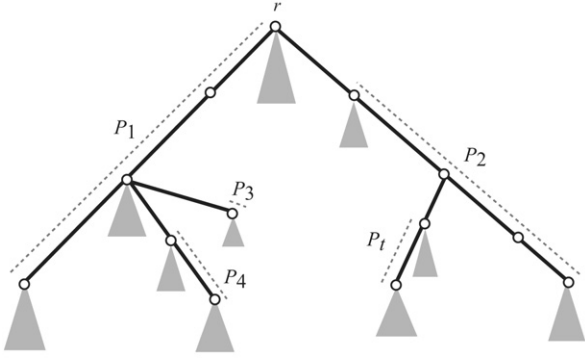
Then we easily observe the following property.

Lemma 4. For an edge-weighted rooted tree (T, w) , let (T', w) be an edge-weighted rooted tree obtained from T by applying a *lifting* or *splitting* operation. Then $w_{\text{max}}^*(T', w', p) \leq w_{\text{max}}^*(T, w, p)$ holds for any integer $p \geq 1$, and $p^*(T', w', x) \leq p^*(T, w, x)$ and $w_{\text{sum}}^*(T', w', x) \leq w_{\text{sum}}^*(T, w, x)$ hold for any real $x > 0$.

For a given real $x > 0$ and an edge-weighted rooted tree (T, w) , a vertex v is called *heavy* if

$$w(T\langle D(\{v\}) \rangle) \geq x - d(v)$$

holds, or *light* otherwise. An edge $e = (u, v)$ with $v \in \text{Ch}(u)$ is called *heavy* if v is heavy. Then the set of heavy

Fig. 1. Thick lines show heavy edges in a rooted tree T .

edges induces from T a subtree containing r , which we call the *heavy tree* and denote it by T^* (see Fig. 1).

Given an edge-weighted rooted tree (T, w) and a real $x > 0$, the following algorithm converts the tree (T, w) into another tree (T', w') by applying lifting and splitting operations, during which a partition $\mathcal{L}$ of the set $L(T)$ of leaves in T and an independent subpartition $\mathcal{L}'$ of the set $L(T')$ of leaves in T' are constructed. An approximate solution is constructed from the partition $\mathcal{L}$ of $L(T)$, and a lower bound on $p^*(T, w, x)$ is computed from the independent subpartition $\mathcal{L}'$ of $L(T')$.

Algorithm PARTITION

Input: An edge-weighted rooted tree (T, w) and a real $x \geq \max_{u \in L(T)} d(u)$.

Output: A partition $\mathcal{L}$ of $L(T)$, a subtree T' obtained by applying lifting and splitting operations, and an independent subpartition $\mathcal{L}'$ of $L(T')$.

Step 1. Let $\mathcal{L} := \mathcal{L}' := \emptyset$. For each heavy vertex u , we partition the set of light children of u into $C_0, C_1, C_2, \dots, C_t$ such that

$$x - d(u) \leq w(T \langle D(C_i) + u \rangle) \leq 2(x - d(u)),$$

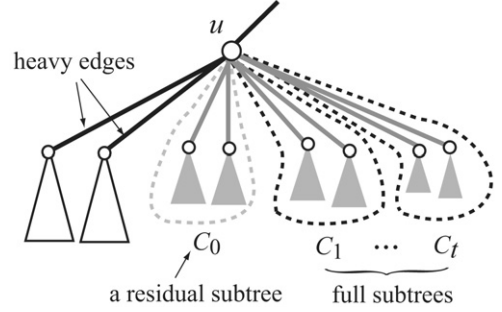
$$i = 1, 2, \dots, t$$

and

$$w(T \langle D(C_0) + u \rangle) < x - d(u),$$

where we can find such a partition $C_0, C_1, \dots, C_t$ since each light child $v \in Ch(u)$ satisfies $w(T \langle D(\{v\}) + u \rangle) < x - d(u)$. Call the subtree $T \langle D(C_i) + u \rangle$, $i \geq 1$ *full*, and the subtree $T \langle D(C_0) + u \rangle$ with $C_0 \neq \emptyset$ *residual* (see Fig. 2).

Step 2. Consider the heavy tree T^* of T . Let $t = |L(T^*)|$. We remove $t - 1$ edges from T^* so that the resulting forest consists of t paths $P_1, P_2, \dots, P_t$ such that each P_i connects a leaf of T^* and an ancestor of the leaf (see Fig. 1).

Fig. 2. A partition of the set of light children of a vertex u .

For each path P_i , we execute the following Steps 2a–2c.

Step 2a. We consider the residual subtrees $ST_1, ST_2, \dots, ST_q$ incident to P_i , where the root of ST_i is denoted by r_i and r_i is an ancestor of r_{i+1} . We partition the set of these subtrees into $\mathcal{T}_1 = \{ST_1 = ST_{i_1}, ST_2, \dots, ST_{i_2-1}\}$, $\mathcal{T}_2 = \{ST_{i_2}, \dots, ST_{i_3-1}\}$, $\dots$, $\mathcal{T}_j = \{ST_{i_j}, \dots, ST_{i_{j+1}-1}\}$, $\dots$, $\mathcal{T}_k = \{ST_{i_k}, \dots, ST_q\}$ such that

$$x - d(r_{i_j}) \leq w(\mathcal{T}_j) \leq x - d(r_{i_j}) + x - d(r_{i_{j+1}-1}),$$

$$1 \leq j \leq k - 1,$$

and

$$w(\mathcal{T}_k) \leq x - d(r_{i_k}) + x - d(r_q),$$

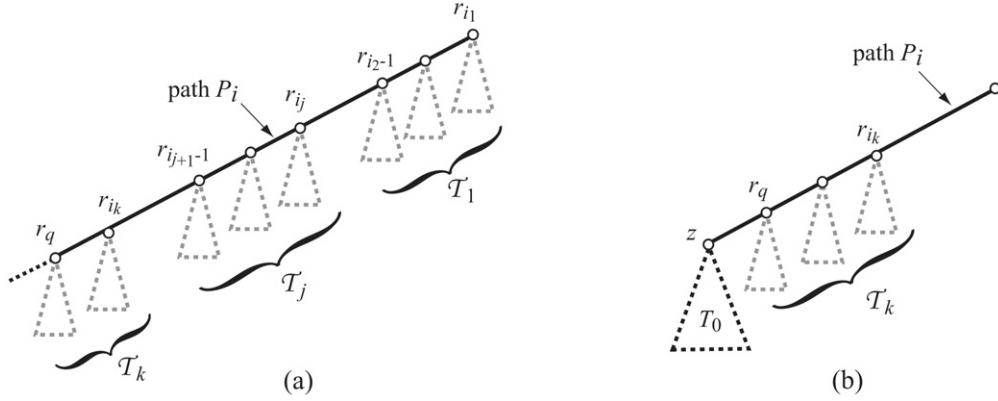
where $w(\mathcal{T}_j)$ denotes $\sum_{ST_i \in \mathcal{T}_j} w(ST_i)$. See Fig. 3(a). (We can find such a partition $\mathcal{T}_1, \dots, \mathcal{T}_k$ since $w(ST_j) < x - d(r_{i_j})$ holds for all j .) For $j = 1, 2, \dots, k$, let L_j be the set of all leaves in the subtrees in $\mathcal{T}_j$.

Step 2b. For each $\mathcal{T}_j$, $j = 1, 2, \dots, k - 1$, we lift up subtrees in $\mathcal{T}_j$ at the vertex r_{i_j} and let $\mathcal{L} := \mathcal{L} \cup \{L_j\}$ and $\mathcal{L}' := \mathcal{L}' \cup \{L_j\}$. If $x - d(r_{i_k}) \leq w(\mathcal{T}_k)$, then we also lift up subtrees in $\mathcal{T}_k$ to the vertex r_{i_k} , letting $\mathcal{L} := \mathcal{L} \cup \{L_k\}$ and $\mathcal{L}' := \mathcal{L}' \cup \{L_k\}$. Otherwise (if $w(\mathcal{T}_k) < x - d(r_{i_k})$) execute the next step for treating $\mathcal{T}_k$.

Step 2c. Let z be the end vertex of P_i that is a leaf of T^* , where possibly $z = r_q$ (see Fig. 3(b)). Since the edge between z and its parent is heavy, there is a full subtree T_0 rooted at z such that $x - d(z) \leq w(T_0) < 2(x - d(z))$. Let L_0 be the set of leaves in the subtree in T_0 .

Case 1: $w(T_0) + w(\mathcal{T}_k) < x - d(r_{i_k})$. Let $\mathcal{L} := \mathcal{L} \cup \{L_k \cup L_0\}$ and $\mathcal{L}' := \mathcal{L}' \cup \{L_0\}$ (we do not put $L_k \cup L_0$ into $\mathcal{L}'$ so that $\text{lca}(L_0) = z$ holds).

Case 2: $x - d(r_{i_k}) \leq w(T_0) + w(\mathcal{T}_k) < 2x - d(z) - d(r_{i_k})$. We lift up subtrees in $\mathcal{T}_k \cup \{T_0\}$ to the vertex

Fig. 3. Illustration for residual subtrees on a path P_i .

r_{ik} . Let $\mathcal{L} := \mathcal{L} \cup \{L_0 \cup L_k\}$ and $\mathcal{L}' := \mathcal{L}' \cup \{L_0 \cup L_k\}$.

Case 3: $w(T_0) + w(T_k) \geq 2x - d(z) - d(r_{ik})$. Let $\mathcal{L} := \mathcal{L} \cup \{L_0, L_k\}$. By applying lifting and splitting operations, we convert T_0 into two subtrees T'_0 and T''_0 rooted at z such that $w(T'_0) = x - d(z)$ and $w(T''_0) = w(T_0) - x + d(z)$. Then we lift up all subtrees in $T_k \cup \{T''_0\}$ to r_{ik} . For the set L'_0 of leaves in T'_0 and the set L'_k of leaves in the subtrees in $T_k \cup \{T''_0\}$, let $\mathcal{L}' := \mathcal{L}' \cup \{L'_0, L'_k\}$.

Step 3. For each of remaining full subtrees ST that are not processed in Step 2c, let L be the set of leaves in ST , and $\mathcal{L} := \mathcal{L} \cup \{L\}$ and $\mathcal{L}' := \mathcal{L}' \cup \{L\}$.

It is not difficult to see that the above algorithm PARTITION can be implemented to run in linear time.

Lemma 5. For a given tree (T, w) with a set $L(T)$ of leaves and a real $x > \max\{d(u) \mid u \in L(T)\}$, let (T', w') , $\mathcal{L}$ and $\mathcal{L}'$ be, respectively, a tree, a partition of $L(T)$ and a subpartition of $L(T')$ obtained from (T, w) and x by PARTITION. Then

- (i) For each $L \in \mathcal{L}$, $w(T\langle L + r \rangle) \leq 2x$.
- (ii) For each $L' \in \mathcal{L}'$, $w'(T'\langle L' + lca(L') \rangle) \geq x - d'(lca(L'))$, where $d'(u) = d_{T'}(r, u)$.
- (iii) $|\mathcal{L}| \leq p^*(T, w, x)$.
- (iv) $w_{\text{sum}}(\mathcal{L}) \leq 2w_{\text{sum}}^*(T, w, x)$.

Proof. (i) It suffices to show that $w(T\langle L + lca(L) \rangle) \leq 2x - d(lca(L))$, by which $w(T\langle L + r \rangle) \leq w(T\langle L + lca(L) \rangle) + d(lca(L)) \leq 2x - d(lca(L)) + d(lca(L)) = 2x$ holds. If L is added to $\mathcal{L}$ in Case 3 of Step 2c or in Step 3, then $T\langle L + lca(L) \rangle$ is a subtree constructed in Step 1 and thereby $w(T\langle L + lca(L) \rangle) \leq 2(x - d(lca(L)))$ holds. Similarly for the case that L

is added to $\mathcal{L}$ in Step 2b or in Cases 1–2 of Step 2c, where $w(T\langle L + lca(L) \rangle) \leq 2x - d(lca(L))$ holds.

(ii) We only consider the case where L' is added to $\mathcal{L}'$ in Case 3 (for other cases we easily see that $w'(T'\langle L' + lca(L') \rangle) \geq x - d'(lca(L'))$ holds since L' contains a full subtree). If $L' = L'_0$ in Case 3, then $w'(T'\langle L' + lca(L') \rangle) = w(T'_0) = x - d(z) = x - d'(lca(L'))$ holds. If $L' = L'_k$ in Case 3, then $w'(T'\langle L' + lca(L') \rangle) \geq w(T_k) + w(T''_0) = w(T_k) + w(T_0) - x + d(z) \geq (2x - d(z) - d(r_{ik})) - x + d(z) = x - d(r_{ik}) = x - d'(lca(L'))$ holds by $w(T_0) + w(T_k) \geq 2x - d(z) - d(r_{ik})$.

(iii) Obviously $|\mathcal{L}| = |\mathcal{L}'|$ and $\mathcal{L}'$ is an independent subpartition of $L(T')$. From property (ii), it follows $|\mathcal{L}'| \leq \hat{p}(\mathcal{L}', T', w')$. By Corollary 3(ii), we have $\hat{p}(\mathcal{L}', T', w') \leq p^*(T', w', x)$. Since (T', w') is obtained from (T, w) by lifting and/or splitting operations, it holds $p^*(T', w', x) \leq p^*(T, w, x)$ by Lemma 4. Therefore, $|\mathcal{L}| = |\mathcal{L}'| \leq \hat{p}(\mathcal{L}', T', w') \leq p^*(T', w', x) \leq p^*(T, w, x)$, proving (iii).

(iv) It follows from $|\mathcal{L}| = |\mathcal{L}'|$ and property (i) that $w_{\text{sum}}(\mathcal{L}) \leq 2x|\mathcal{L}| = 2x|\mathcal{L}'|$. From property (ii), it follows $x|\mathcal{L}'| \leq x\hat{p}(\mathcal{L}', T', w')$. By Corollary 3(i), we have $x\hat{p}(\mathcal{L}', T', w') \leq w_{\text{sum}}^*(T', w', x)$. Since $w_{\text{sum}}^*(T', w', x) \leq w_{\text{sum}}^*(T, w, x)$ by Lemma 4, it follows that $w_{\text{sum}}(\mathcal{L}) \leq 2x|\mathcal{L}| = 2x|\mathcal{L}'| \leq x\hat{p}(\mathcal{L}', T', w') \leq w_{\text{sum}}^*(T', w', x) \leq w_{\text{sum}}^*(T, w, x)$. $\square$

From Lemma 5 (i), (iii) and (iv), we see that the BRSC is 2-approximable in the following sense.

Theorem 6. For a given instance (T, w, x) of the BRSC, a p -partition $\mathcal{L}$ of $L(T)$ satisfying $p \leq p^*(T, w, x)$, $w_{\text{max}}(\mathcal{L}) \leq 2x$, $w_{\text{sum}}(\mathcal{L}) \leq 2w_{\text{sum}}^*(T, w, x)$ can be found in $O(n)$ time.

Finally we prove that the MRSC is $(2 + \varepsilon)$ -approximable for any $\varepsilon > 0$.

Theorem 7. For a given instance $I = (T, w, p)$ of the MRSC and a specified $\varepsilon > 0$, a p -partition $\mathcal{L}$ of $L(T)$ satisfying $w_{\max}(\mathcal{L}) \leq (2 + \varepsilon)w_{\max}^*(I)$ can be found in $O(n \log \log_{1+\varepsilon/2} 3)$ time.

Proof. For a specified $\varepsilon > 0$, we find two reals $x_1, x_2 \in [LB, 3LB]$ with $x_1 < x_2 \leq (1 + \varepsilon/2)x_1$ such that the partitions $\mathcal{L}_1$ and $\mathcal{L}_2$ of $L(T)$ computed by PARTITION for x_1 and x_2 , respectively satisfy $|\mathcal{L}_2| \leq p < |\mathcal{L}_1|$. Such x_1 and x_2 can be found by conducting a binary search over the range $[LB, 3LB]$, during which PARTITION is invoked $O(\log t)$ times for the integer $t = \lceil \log_{1+\varepsilon/2} 3 \rceil$. It follows from Lemma 5(iii) that $p < |\mathcal{L}_1| \leq p^*(T, w, x_1)$, indicating that $w_{\max}^*(I) > x_1$. By $|\mathcal{L}_2| \leq p$ and Lemma 5(i), each $L_i \in \mathcal{L}_2$ satisfies $w(T \langle L_i + r \rangle) \leq 2x_2 \leq 2(1 + \varepsilon/2)x_1 < (2 + \varepsilon)w_{\max}^*(I)$. Hence $w_{\max}(\mathcal{L}_2) \leq (2 + \varepsilon)w_{\max}^*(I)$ holds. $\square$

Acknowledgement

The authors would like to thank anonymous referees for their helpful comments. This research was partially supported by the Scientific Grant-in-Aid from Ministry of Education, Science, Sports and Culture of Japan.

References

- [1] I. Averbakh, O. Berman, A heuristic with worst-case analysis for minmax routing of two traveling salesmen on a tree, *Discrete Appl. Math.* 68 (1996) 17–32.
- [2] J. Bruno, P. Downey, Complexity of task sequencing with deadlines, set-up times and changeover costs, *SIAM J. Comput.* 7 (1978) 393–581.
- [3] G. Even, N. Garg, J. Könemann, R. Ravi, A. Sinha, Min-max tree covers of graphs, *Oper. Res. Lett.* 32 (2004) 309–315.
- [4] M.R. Garey, D.S. Johnson, *Computers and Intractability; A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, 1979.
- [5] Y. Karuno, H. Nagamochi, 2-approximation algorithms for the multi-vehicle scheduling on a path with release and handling times, *Discrete Appl. Math.* 129 (2003) 433–447.
- [6] Y. Karuno, H. Nagamochi, Scheduling vehicles on trees, *Pacific J. Optim.* 1 (2005) 527–543.
- [7] G. Laporte, Location-routing problems, in: B.L. Golden, A.A. Assad (Eds.), *Vehicle Routing: Methods and Studies*, North-Holland, Amsterdam, 1988, pp. 163–197.
- [8] H. Nagamochi, Approximating the minmax rooted-subtree cover problem, *Electron. Inform. Comm. Eng. Trans. Fundamentals E88-A* (2005) 1335–1338.
- [9] H. Nagamochi, K. Okada, Polynomial time 2-approximation algorithms for the minmax subtree cover problem, in: *The 14th Annual International Symposium on Algorithms and Computation (ISAAC2003)* Kyoto, December 15–17, in: *Lecture Notes in Computer Science*, vol. 2906, Springer-Verlag, 2003, pp. 138–147.